Art Gallery
Requirements and Secure Design

BIBIFI Group 11 | Huseyin Simsek and Gustavo Pepe
Weeks 4–6 · Secure Art Gallery · Fall 2026

What we are building

Our project is a web application that keeps track of people moving through an art gallery: who came into the building, which rooms they went into, and who is inside right now. Guests, employees and administrators each see a different slice of that information, and every rule about who can see or change what is enforced by the server, not by the page.

Team, repository and tools

Team: Huseyin Simsek and Gustavo Pepe (Group 11)
Repository: github.com/hsimsek1/art-gallery-group11

We use Python 3.12 and Flask for the backend, with HTML/CSS and Jinja for the server-rendered frontend. Data is stored in SQLite through Python's built-in sqlite3 module. Werkzeug, which ships with Flask, hashes passwords; Flask-Login manages sessions; Flask-WTF adds CSRF protection. Private settings are loaded from instance/config.py, which never goes into Git.

Who uses the system

We keep Users (accounts that log in) separate from Persons (people whose movements are recorded). Not every Person needs an account, and keeping the two apart is what will later let a guest see their own visits and nobody else's.

Role	What it can do today	What it will do in the finished app
Guest	Log in, see the room list, log out.	See their own gallery history and the information they are allowed to view.
Employee	Same as a guest for now.	Record gallery and room entries and exits; view occupancy and history.
Administrator	Everything above, plus the protected admin page.	Manage accounts and roles; read the audit log and other administrative information.

Functional requirements

ID	Requirement	Status
FR-1	Create and initialize a persistent SQLite database for the application's data.	Done | Week 6
FR-2	Seed one login account per role plus fictional persons and rooms. Running the setup again must keep existing records.	Done | Week 6
FR-3	Authenticate a user by username and password and reject wrong credentials.	Done | Week 6
FR-4	Keep a signed login session and let the user log out.	Done | Week 6
FR-5	Require login before showing room information from SQLite.	Done | Week 6

ID	Requirement	Status
FR-6	Let only an administrator reach administrator functions.	Done | Week 6
FR-7	Record a person entering the gallery.	Planned | Week 7
FR-8	Record a person leaving the gallery.	Planned | Week 7
FR-9	Record a person entering or leaving a specific room.	Planned | Week 7
FR-10	Show who is currently in the gallery and in each room.	Planned | Week 7
FR-11	Search or query past gallery events.	Planned | Week 7
FR-12	Provide administrative functions, including account and role management.	Planned | Week 7
FR-13	Reject invalid or unauthorized movements, including impossible and duplicate events (rules below).	Planned | Week 7

Movement rules

1. Entering the gallery is only allowed from outside.
2. Entering a room requires being in the gallery and not already in a room, and the room has to exist.
3. Leaving a room is only allowed for the room the person is actually in.
4. Leaving the gallery requires being in the gallery and not inside any room.
5. An event that breaks a rule is answered with **409**, written to the audit log as rejected, and changes nothing else.
6. The rule check and the insert run inside one BEGIN IMMEDIATE transaction, so two requests sent at the same instant cannot both pass the check.

Security requirements

These are the security requirements we started from. The ones marked are already in the Week 6 code; the rest are tied to the milestone where the feature they protect gets built. SEC-13 to SEC-15 came out of our STRIDE work in the design document.

ID	Requirement	Status
SEC-1	Store salted password hashes using Werkzeug scrypt. Never store plain text passwords.	Done
SEC-2	Verify credentials before login and use one generic “invalid credentials” message.	Done
SEC-3	Enforce login and administrator permissions in Flask routes; hiding links in the page is not a control.	Done
SEC-4	Use Flask-Login signed sessions with HttpOnly, SameSite=Lax cookies and a 30-minute lifetime; clear the session at login and logout.	Done
SEC-5	Validate login input on the server and require CSRF tokens on state-changing forms.	Done
SEC-6	Use parameterized sqlite3 queries for variable values and rely on Jinja escaping for displayed text.	Done
SEC-7	Generate local secrets and seed passwords outside Git; exclude local config, credentials, databases and the virtual environment.	Done

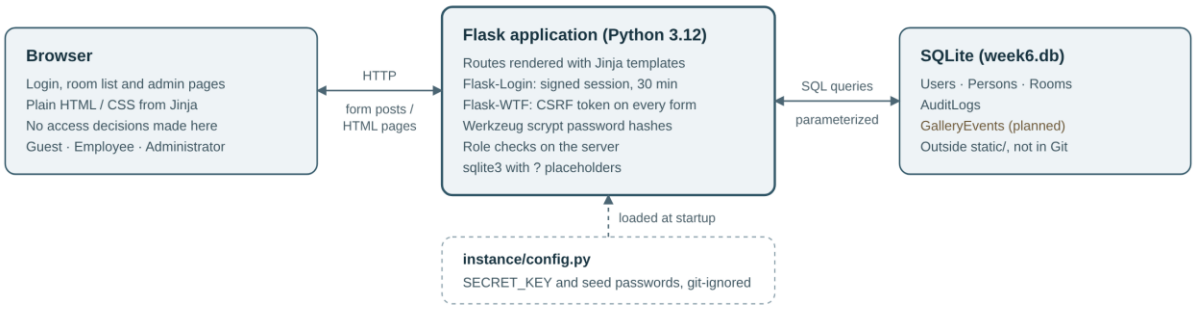
ID	Requirement	Status
SEC-8	Log login success, login failure and logout with time and user ID when known; never log passwords or tokens.	Done
SEC-9	Limit request bodies to 16 KiB as a basic availability safeguard.	Done
SEC-10	Enforce authorization and valid state transitions for gallery and room movements.	Planned | Week 7
SEC-11	Prevent unauthorized record access (IDOR) in history and administrative functions.	Planned | Week 8
SEC-12	Address XSS, authentication attacks, privilege escalation, session attacks, information disclosure, secure error handling and misconfiguration.	Planned | Week 8
SEC-13	Rate-limit the login form and lock an account for a period after repeated failed attempts.	Planned | Week 8
SEC-14	Require pagination with a fixed maximum page size on history queries.	Planned | Week 7
SEC-15	Extend the audit log to accepted and rejected events, every 403 and every admin change, with actor, target, outcome and IP.	Planned | Weeks 7–8

What “secure” means for this project

Password material never leaves the server in usable form, and each role sees only what it is allowed to see. Roles are checked on the server, every query is parameterized, and gallery state only changes through valid events. Request bodies are capped, and history queries will be bound. Logins and logouts are logged today; gallery events and admin actions will join the same log.

Architecture

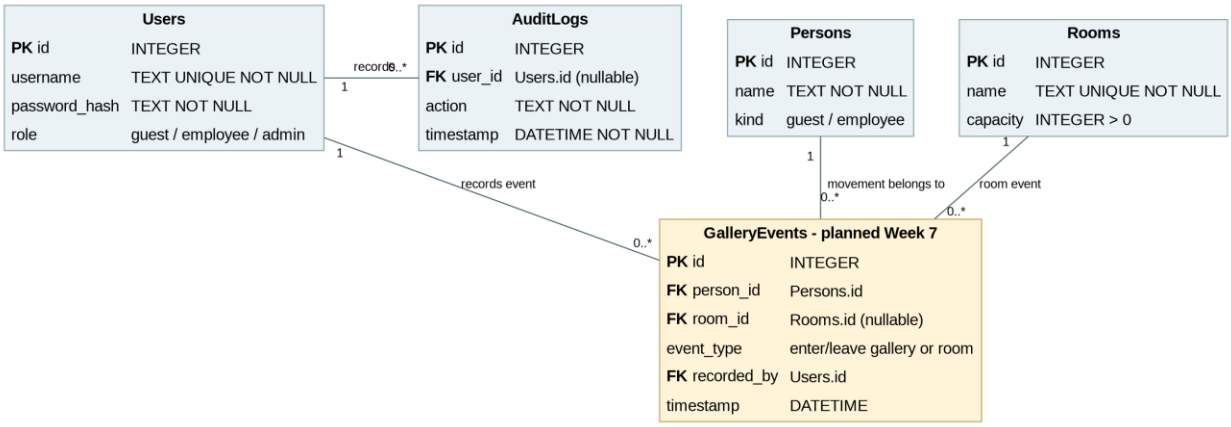
The whole system is one Flask application. It renders pages with Jinja, checks logins and roles, and reads and writes SQLite through Python’s sqlite3 module. With one server and server-rendered pages, a separate REST API would only add another surface to defend, so we left it out. The design still has room for the gallery-event operations that come in the later Build It weeks.



Database and ER Digram

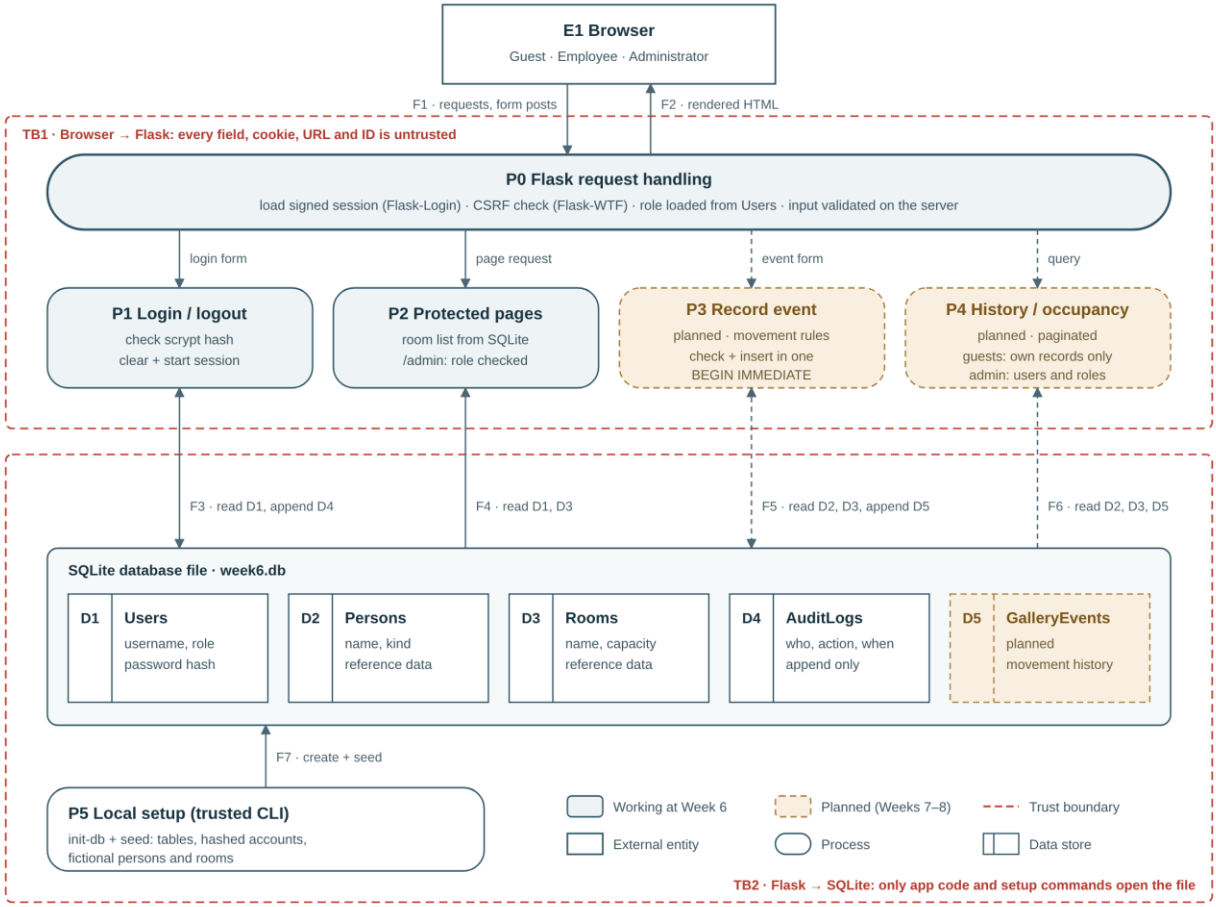
A User is a login account, while a Person is someone whose movements are recorded. They remain separate because not every Person needs an account. The current Week 6 database contains Users, Persons, Rooms and AuditLogs. The Week 5 design adds GalleryEvents for movement history. In that planned flow, current location is reconstructed

from a person's latest valid event rather than stored twice and recorded_by/timestamp are supplied by the server. Flask-Login keeps the session in a signed cookie, so no Sessions table is required.



Data-flow diagram

Requests come in from the browser, pass through one layer of checks in Flask, and only then reach the code that touches the database. The two dashed lines are the places where trust changes.



Flow	What moves and what checks it
F1 / F2	The browser sends login, logout, page and (later) gallery requests; Flask sends back HTML. Every form post passes the CSRF check and server-side validation before a route runs.
F3	Login looks up the user, verifies the scrypt hash, and appends LOGIN_SUCCESS, LOGIN_FAILED or LOGOUT to AuditLogs.
F4	Protected routes load the account's role from Users and read Rooms. The admin page checks that role on the server.
F5 | planned	Recording an event reads the person and room, checks the movement rules against the person's last event, and appends a GalleryEvents row, all in one transaction.
F6 | planned	Occupancy, history and admin views query only the records the caller may see and rebuild gallery state from the events.
F7	Local setup commands create the tables and seed hashed accounts plus fictional persons and rooms.

TB1 | Browser to Flask. Anything that arrives from the browser is treated as hostile: form fields, hidden inputs, cookies, URLs and record IDs. Signed sessions, CSRF tokens, input validation and role checks on the server hold this line. Hiding a link or a button is never counted as a control.

TB2 | Flask to SQLite. Only our application code and the setup commands open the database file. Variable values always go through ? placeholders. The database file and the credentials sit outside Sstatic/ and are excluded from Git. SQLite has no database users or permissions of its own, so this boundary depends on our code and on the file permissions of the machine.

The local server uses plain HTTP on 127.0.0.1, so this milestone does not encrypt traffic in transit. We treat the development machine and its file permissions as trusted.

STRIDE threat model

One threat for each STRIDE letter, covering both the login path that runs today and the gallery operations still to be built. For each one: where it hits, how the attack works, what it costs us, and what stands in its way.

STRIDE / Component	Threat scenario	Impact	Mitigation / status
Spoofing Login / session	An attacker guesses an employee/admin password or tries to use a forged or stolen session.	Impersonation and unauthorized access; an attacker could act as another role.	Current: scrypt hashes, generic login errors, random SECRET_KEY outside Git, signed sessions, session cleared at login. Planned (Week 8): rate limiting and temporary lockout.
Tampering Gallery events / state	A user submits fake, duplicate or out-of-order movement events, including concurrent requests that would create an impossible state.	Occupancy and event history become incorrect.	Current: CSRF protects existing state-changing forms. Planned (Week 7): event forms also use CSRF; movement rules are checked server-side and check+insert occurs in one transaction.
Repudiation AuditLogs / events	A user denies logging in/out or an employee later denies recording an event.	Actions cannot be reliably attributed	Current: LOGIN_SUCCESS, LOGIN_FAILED and LOGOUT include timestamp and user ID

STRIDE / Component	Threat scenario	Impact	Mitigation / status
		during review or troubleshooting.	when known. Planned: recorded_by comes from the session and event/admin actions are added to the audit log.
Information Disclosure History / errors / repo	A user changes a record ID to view data they are not allowed to see, an error exposes internals, or a secret is committed.	Privacy breach, exposed credentials or internal implementation details.	Current: secrets excluded from Git, generic login errors, Jinja escaping, DEBUG=False, server-side admin checks. Planned (Weeks 7-8): record-level authorization for history and controlled 404/500 responses.
Denial of Service Flask / history	An attacker sends oversized/repeated requests or asks for an unbounded event history.	The app slows down or becomes unavailable.	Current: request bodies capped at 16 KiB. Planned: bounded/paginated history, rate limiting and availability testing.
Elevation of Privilege Admin operations	A Guest/Employee directly requests an administrator function or submits a role value they are not authorized to set.	Unauthorized account, role or administrative control.	Current: signed sessions, role loaded from Users, login_required and explicit /admin role check. Planned: server-side role checks on every protected admin/gallery operation and allow-listed form fields.

Security checklist

Passwords and login

- Only salted scrypt hashes are stored; passwords are checked with check_password_hash.
- Seed passwords are generated locally by the setup script and never committed.
- Unknown usernames and wrong passwords get exactly the same error.
- Lock an account for a while after repeated failures, and rate-limit the form.

Permissions

- Pages require login_required, and the role comes from the Users table, never from a form field or the cookie.
- The /admin page checks the role on the server.
- role_required on every route, plus a before_request hook that denies anything not on a public allow-list.
- Guests query history only with WHERE person_id = ? bound to their own person.
- User forms copy only allow-listed fields, and an admin cannot change their own role.

SQL injection and XSS

- Variable values go through ? placeholders; no f-strings, % or .format() inside SQL.
- Jinja autoescaping is on for every template.
- Sort columns and directions picked from a fixed allow-list, since placeholders cannot bind column names
- Review that |safe, Markup() and render_template_string are never used on user input.

CSRF

- Flask-WTF CSRF token in every state-changing form.
- Session cookie sent with SameSite=Lax as a second layer.
- Every state change, logout included, is a POST; GET routes never modify data.

Sessions

- Signed Flask-Login session in an HttpOnly cookie with a 30-minute lifetime.
- Session cleared at login (blocks session fixation) and at logout.
- SECRET_KEY generated randomly on each machine and kept in instance/config.py.
- Secure flag on the cookie once the app is served over HTTPS.

Input validation

- Login input validated on the server; request bodies limited to 16 KiB.
- Every gallery form field validated on the server; unexpected or missing fields get a 400.
- HTML required and maxlength used only for convenience, never as the actual check.
- History queries paginated with a maximum page size.

Secrets

- .gitignore excludes local config, credentials, database files and the virtual environment.
- The SQLite file sits outside static/.
- DEBUG off for anything other than local development; scan the Git history for secrets before submitting.

Logging

- Login success, failure and logout recorded with time and user ID; passwords, cookies and tokens never logged.
- Extend the log to accepted and rejected events, every 403 and every admin change, with target, outcome and IP.

Movement rules

The four rules from FR-13 (enter/leave gallery, enter/leave room) enforced on the server. *Week 7*

Check and insert in one BEGIN IMMEDIATE transaction; a rejected event returns 409, and change nothing.

Where the controls live in the code

For anyone reviewing the repository, this is where to find each control today and what the next milestone adds.

Area	In the repository now (Week 6)	Next milestone
Database	database/schema.sql defines Users, Persons, Rooms and AuditLogs; database/seed.sql supplies fictional reference data.	Add GalleryEvents and the relationships movement history needs.
Authentication	src/backend/routes.py verifies passwords with check_password_hash; src/backend/__init__.py seeds scrypt hashes.	Rate limiting and lockout; authentication attack testing.
Sessions / CSRF	src/backend/__init__.py configures Flask-Login, signed sessions, cookie settings, lifetime and Flask-WTF CSRF. Templates include CSRF tokens.	Review session attacks and configuration in the security-engineering milestone.

Area	In the repository now (Week 6)	Next milestone
Authorization	src/backend/routes.py uses login_required and an explicit server-side admin role check.	Role and record-level checks on gallery, history and admin functions.
SQL / output	Variable SQL values use ? binding; Jinja escapes normal output.	SQLi and XSS testing; allow-lists for any dynamic sort or filter input.
Secrets / logging	tools/setup_local.py creates ignored local secrets; .gitignore excludes credentials and databases; db.py records authentication activity.	Audit coverage for gallery events, denials and admin changes.